

CONNECTING GITHUB COPILOT TO KNOWLEDGE AND TOOLS

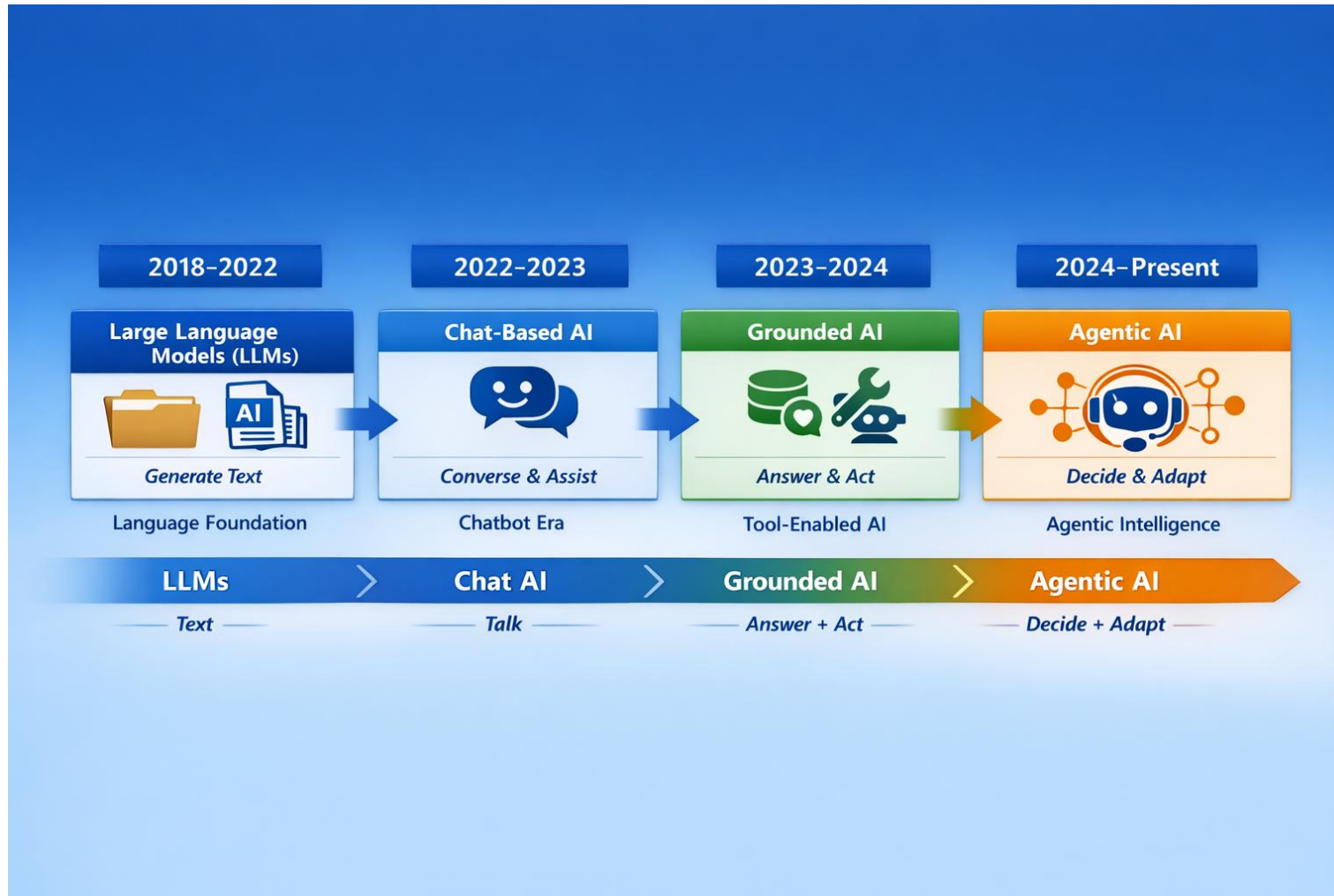
Ronald A. Sumida

ronaldasumida@gmail.com

Agenda

- Context
- Spaces
 - Control what Copilot knows
- Model-Context-Protocol (MCP)
 - Control what Copilot can access and do
 - Connecting to an MCP Server
 - Building an MCP Server
- Tool discovery

Modern AI timeline



Prompt engineering

- GPT models are prompt-based
 - Input text is entered into prompt
 - Response from model represents completion of input
- Results are very dependent upon prompt
 - Obtaining high quality results can be difficult
 - Requires experience and intuition
- Prompt engineering
 - Construct a *single input prompt* to obtain desired response
 - "More art than science"
 - <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering>

Context engineering

- Construct *entire context* so agent can reason toward a desired outcome
 - Instructions
 - State
 - Memory (short and long term)
 - Dynamic knowledge
 - Tools
- Represents the industry's shift from single-prompts->agent-based systems
- Versus prompt engineering
 - Critical technique in the generative AI era
 - Subset of context engineering
 - Prompt engineering is a tactic
 - Context engineering is an architecture

GitHub Copilot Spaces

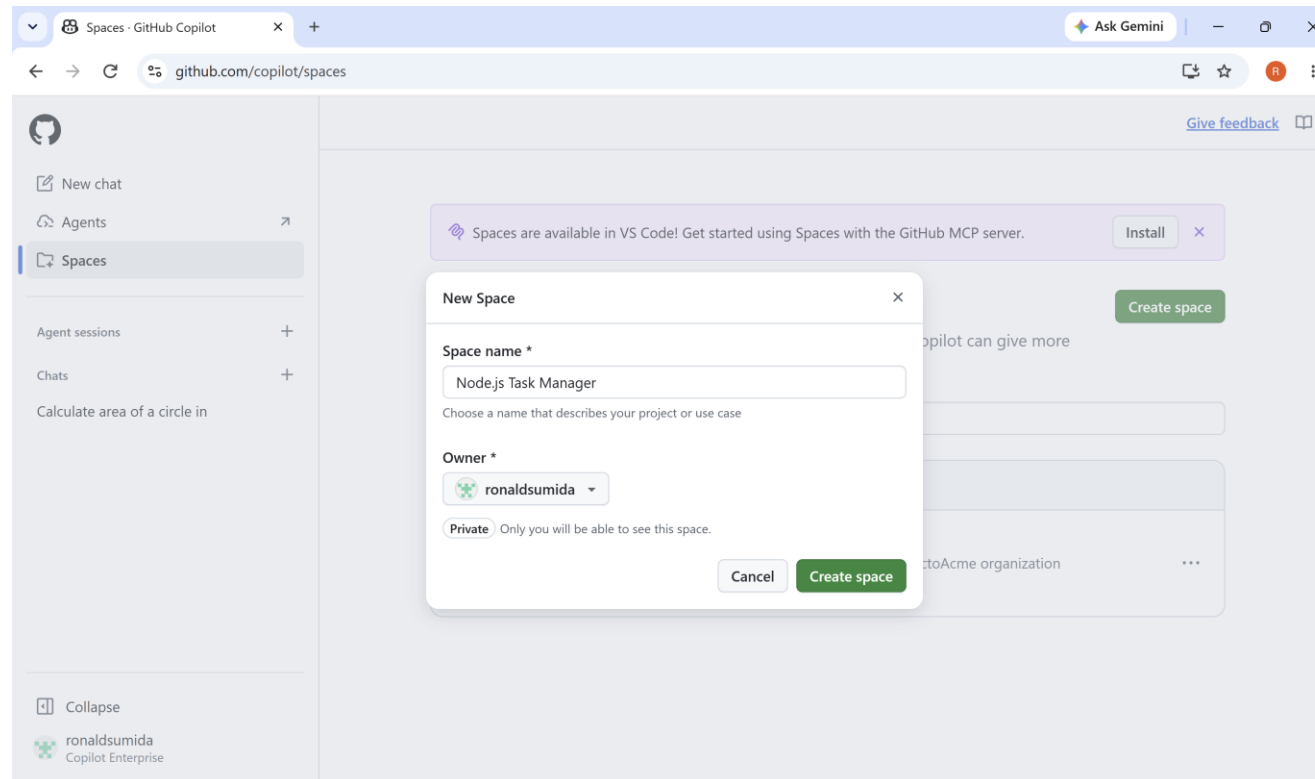
- Curated context for grounding GitHub Copilot Chat responses
 - Files (E.g., code, docs, specs)
 - Issues
 - Pull requests
 - Instructions
- Advantages
 - Centralizes relevant context
 - Enables higher quality responses
 - Allows sharing with collaborators
 - Exists beyond chat history
 - Synchronized to project in GitHub automatically

Availability

- Ownership
 - Organization-owned spaces
 - Can choose access level for members
 - Individual-owned spaces
 - Can be private or shared publicly
- Compatibility
 - Copilot Chat in GitHub
 - <https://github.com/copilot/spaces>,
 - IDE
 - Use GitHub MCP server

Create a new space

- Click *Create space* on <https://github.com/copilot/spaces>
- Example: create new space for Node.js server from *copilot-spaces-demo*

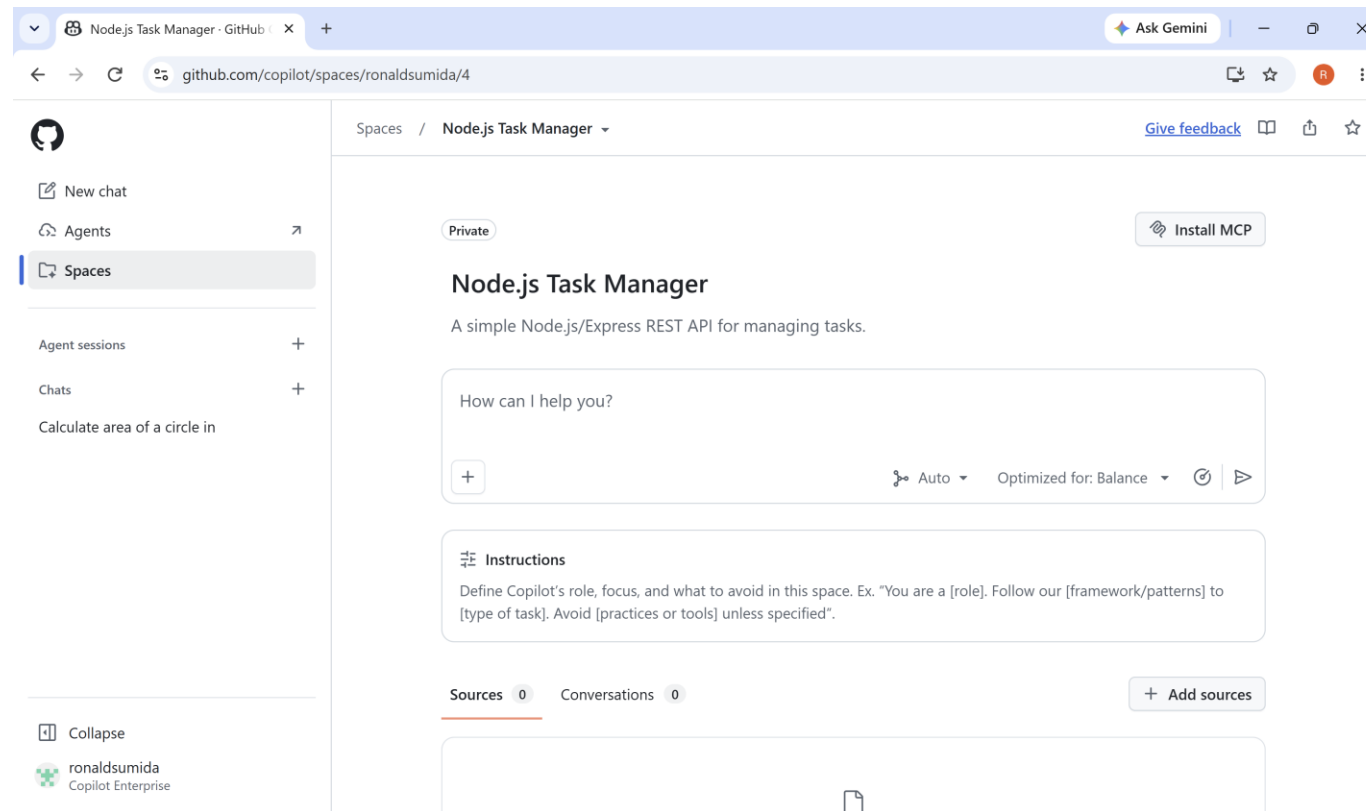


Add context to a new space

- Description
 - Optional
 - Useful for collaborators
 - Not used for responses by Copilot
- Instructions
 - Describe areas of focus within the space
 - E.g., expertise, tasks, dos/don'ts
- Sources
 - Provide context that Copilot will use to provide responses
 - E.g., files, repositories, links, pull requests, issues, text content

Add context to a new space

- Node.js description example (from README.md):
 - *A simple Node.js/Express REST API for managing tasks.*



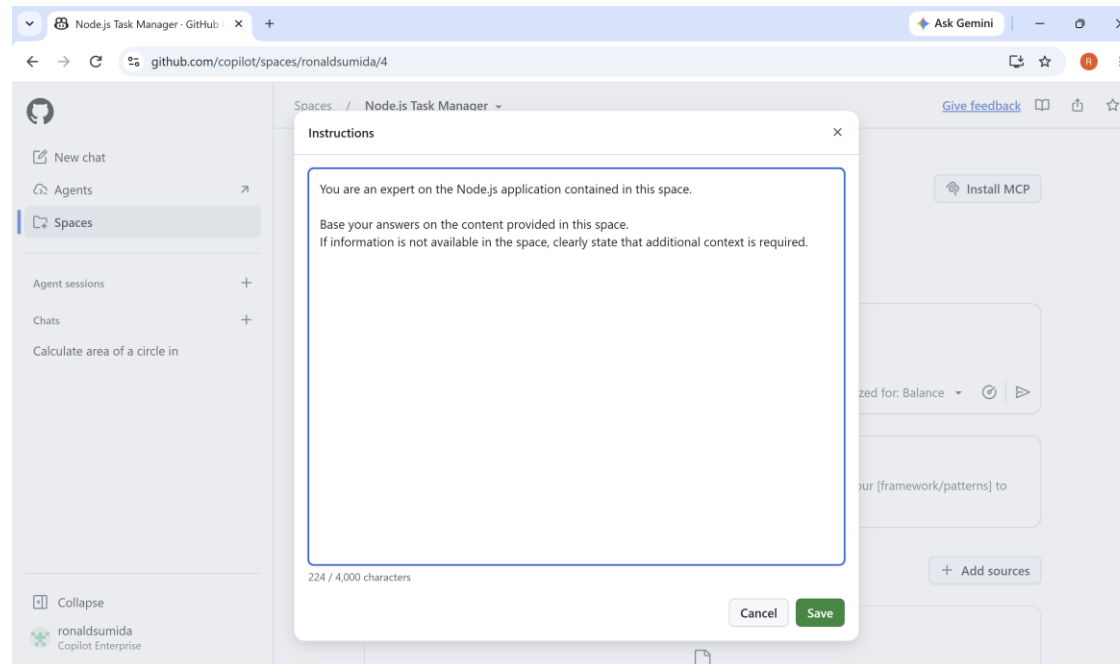
Add instructions to a new space

- Node.js instructions example:

You are an expert on the Node.js application contained in this space.

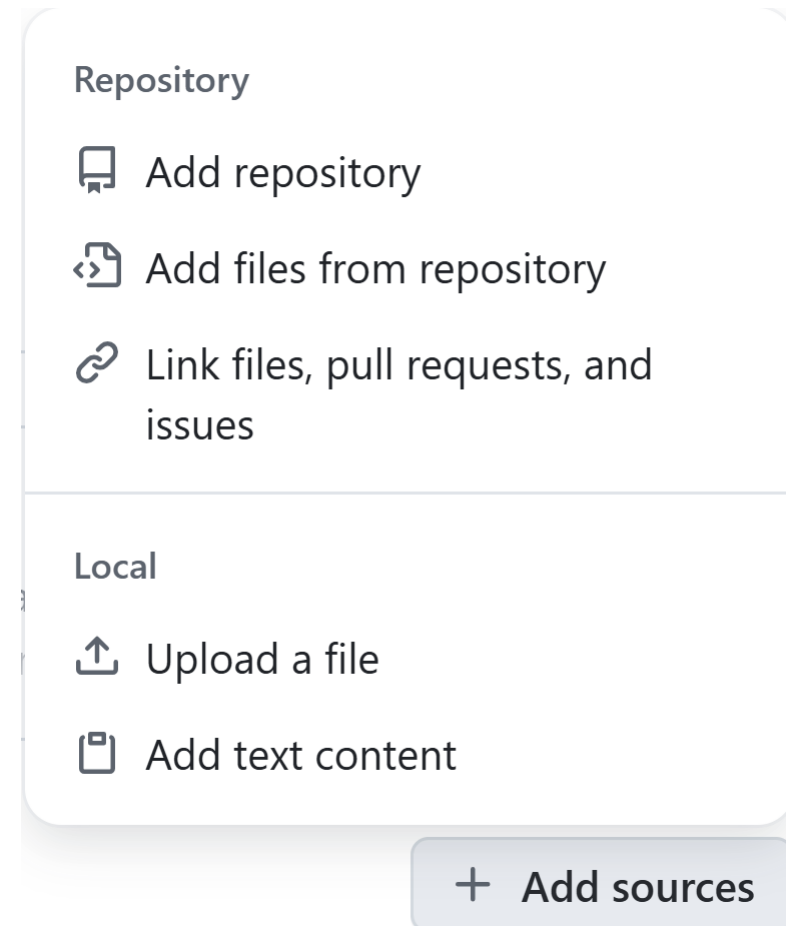
Base your answers on the content provided in this space.

If information is not available in the space, clearly state that additional context is required.



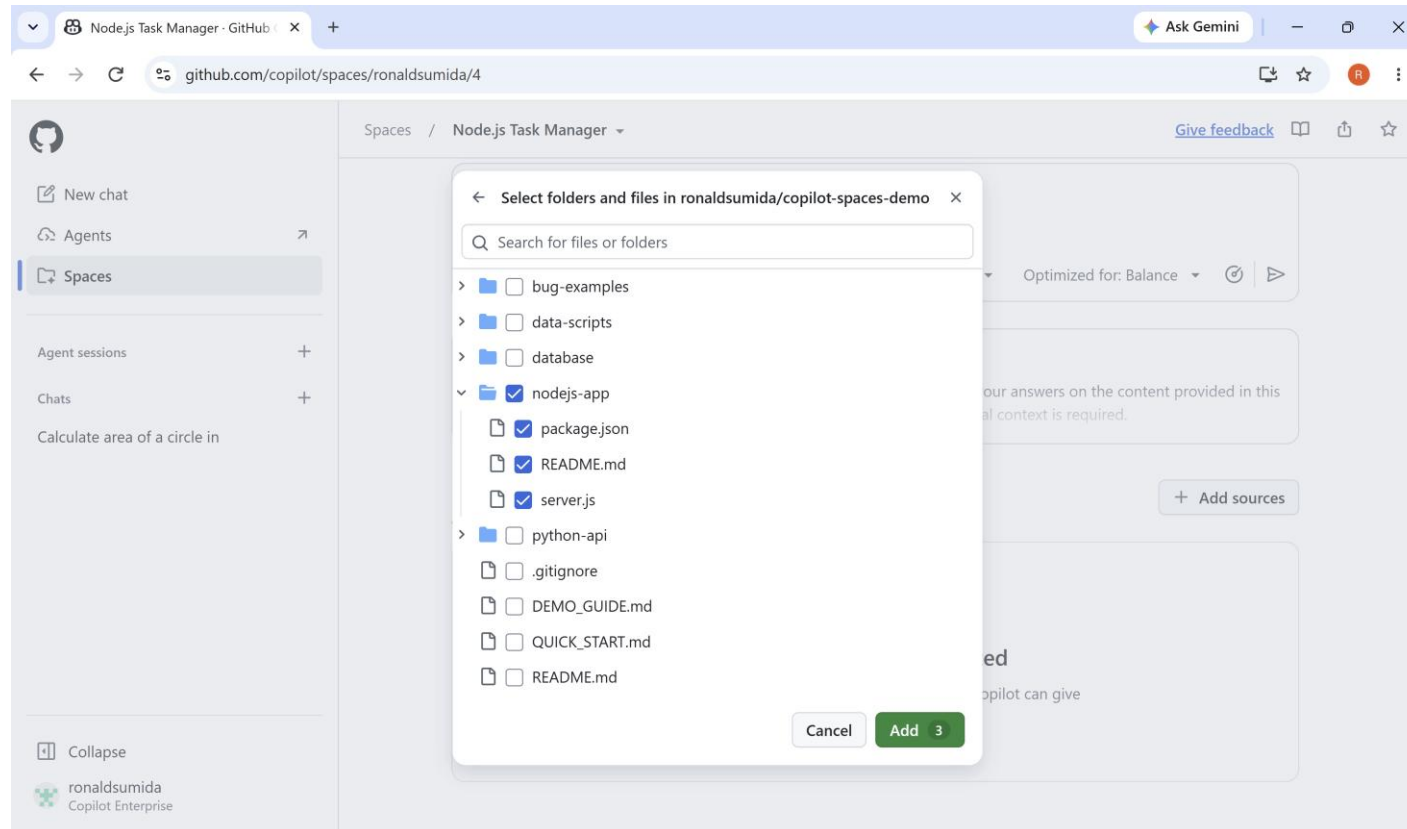
Add sources to a new space

- Repository
 - Doesn't load complete project into memory
 - Searches repository
 - Finds content most relevant to prompt
 - Useful for large-scale scenarios
- Files
 - Complete contents loaded into context window
 - Considered for every prompt
 - Useful for prioritizing particular file(s)



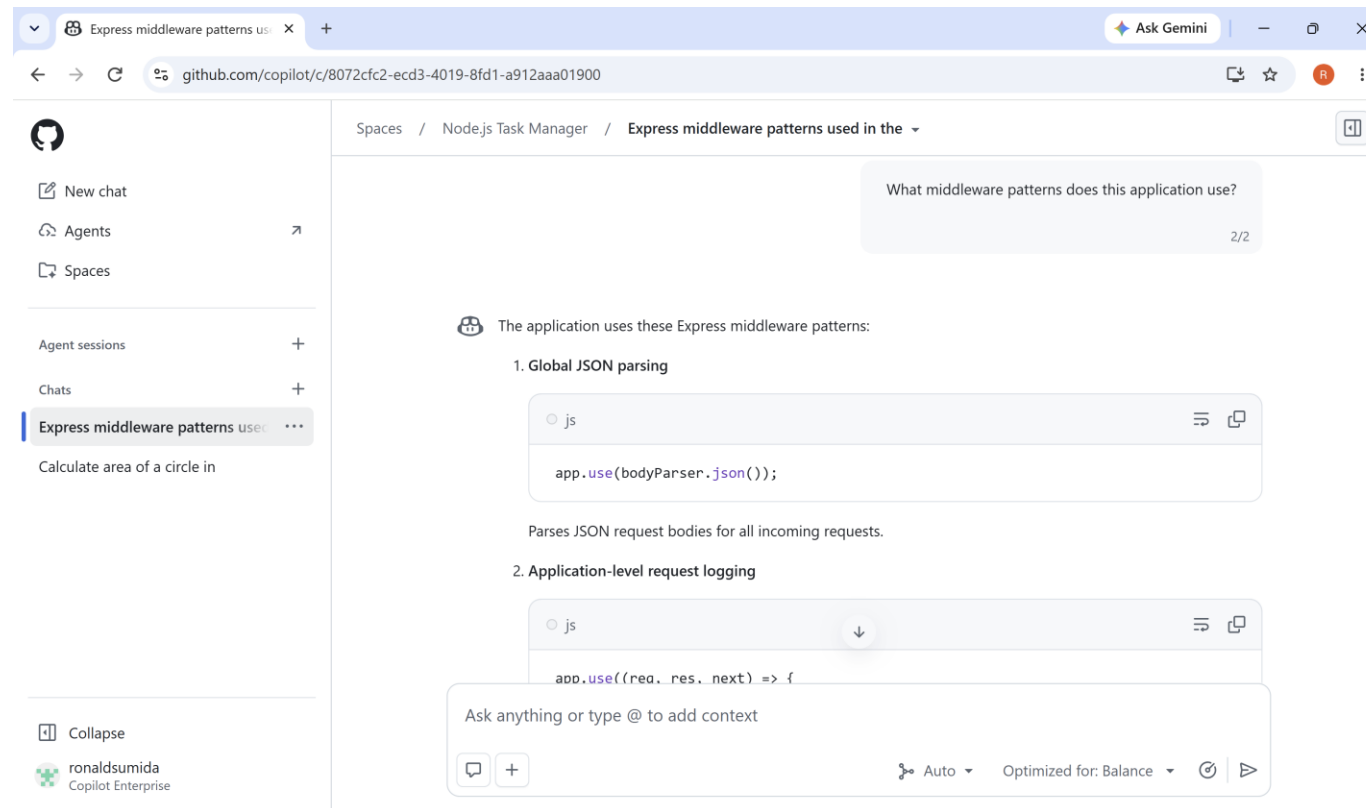
Add sources to a new space example

- Example
 - Click + *Add sources* -> *Add files from repository*



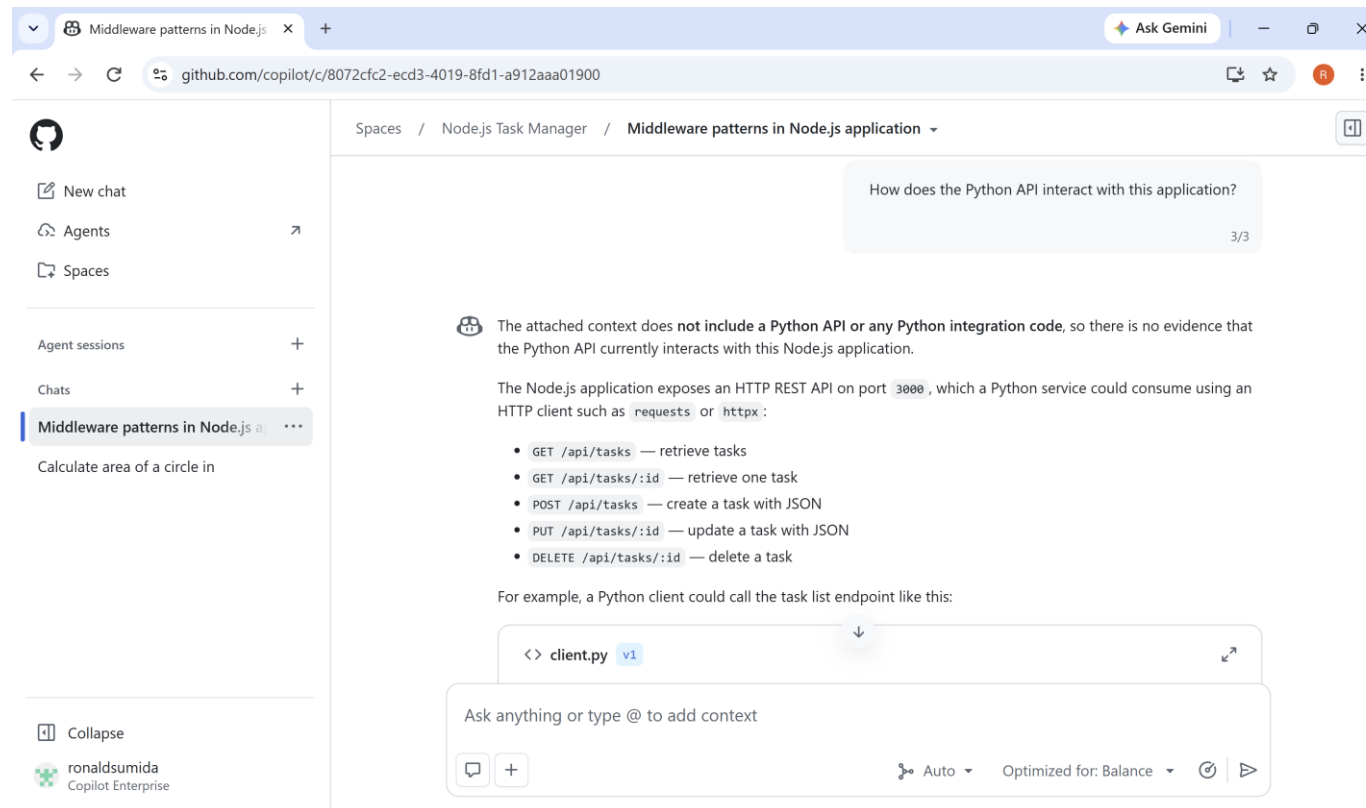
Prompt within context example

- *What middleware patterns does this application use?*
- Responds from included Node.js context



Prompt outside context example

- *How does the Python API interact with this application?*
- Responds that context does not include Python API



Lab 2 – Copilot Spaces

- Scale institutional knowledge using Copilot Spaces
 - <https://github.com/skills/customize-your-github-copilot-experience>

Model Context Protocol (MCP)

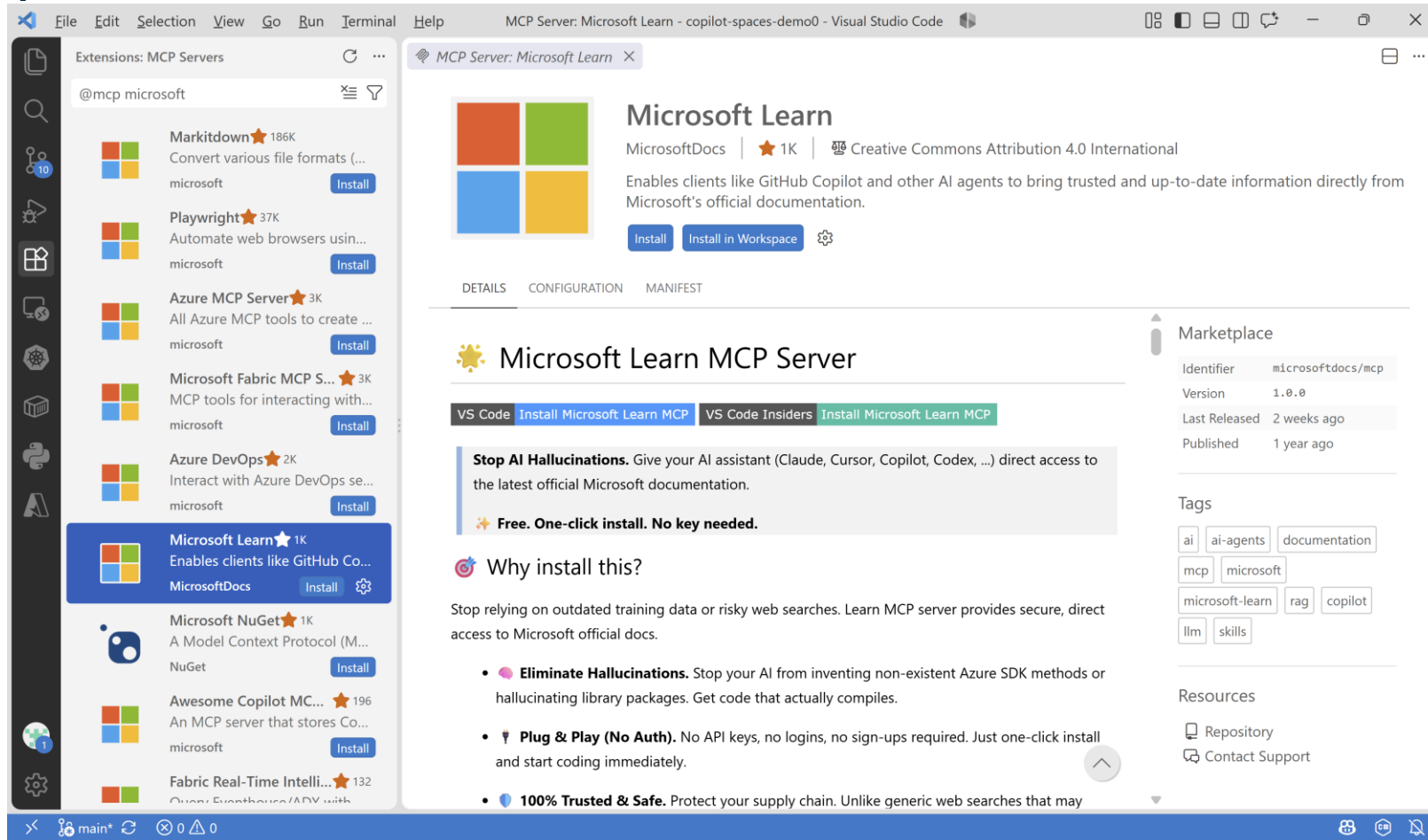
- Open standard for connecting AI systems to tools and data
 - Originally developed by Anthropic
 - Provides context to AI models
 - Enables access to external data sources
- GitHub Copilot uses MCP to:
 - Integrate with tools
 - Retrieve additional context when needed

Connecting to MCP Servers

- MCP servers provide tools to Copilot
 - Expose capabilities (e.g., docs search, APIs, services)
 - Copilot discovers and uses these tools when needed
- Add an MCP server
 - VS Code MCP server gallery
 - Type *@mcp* + <search-term> in the extensions search bar
 - Command palette guided workflow
 - Command Palette -> MCP: Add Server
 - Manually at user or workspace level
 - Directly modify *mcp.json*

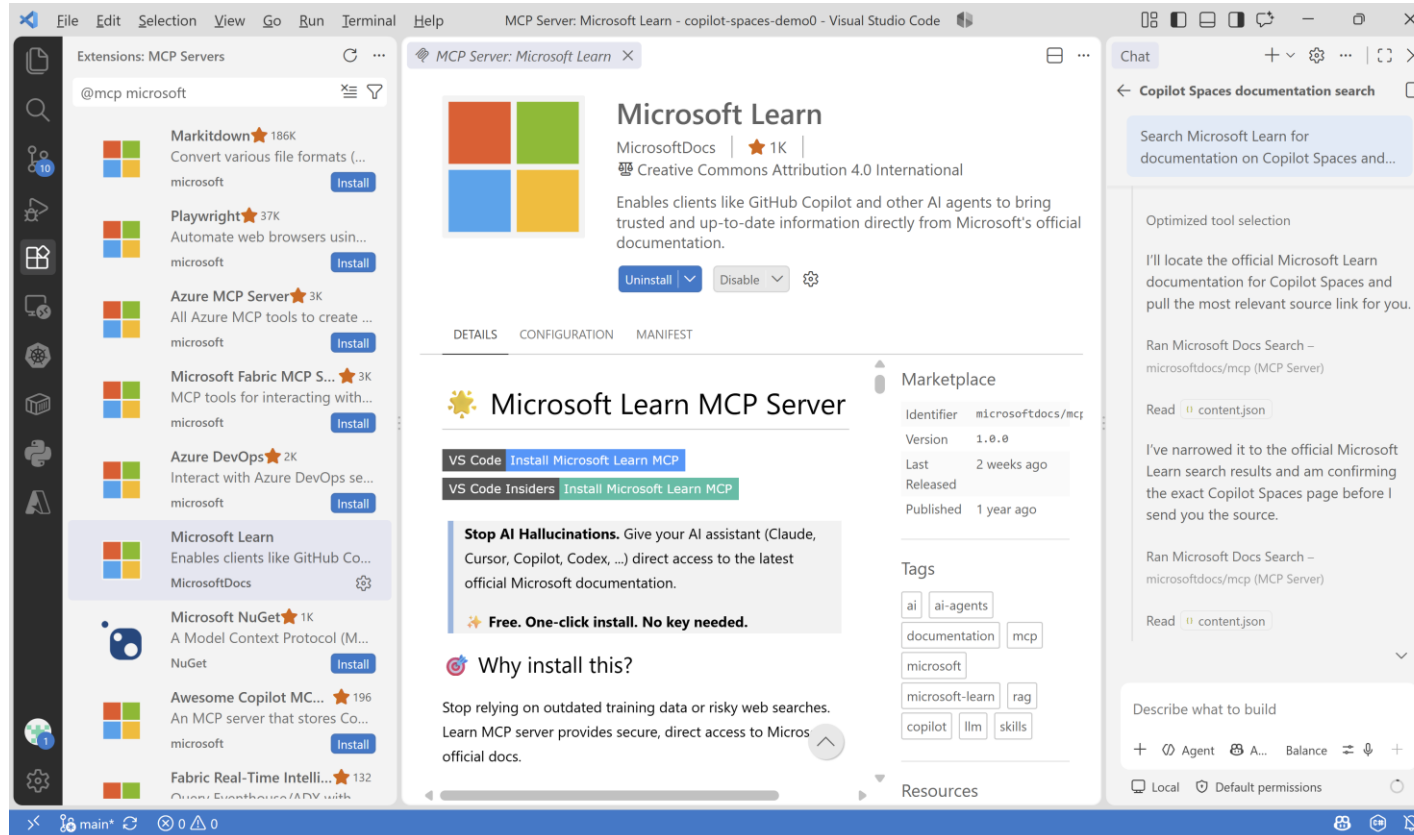
Install from VS Code MCP Server Gallery

- Type `@mcp` + `<search-term>` in the extensions search bar



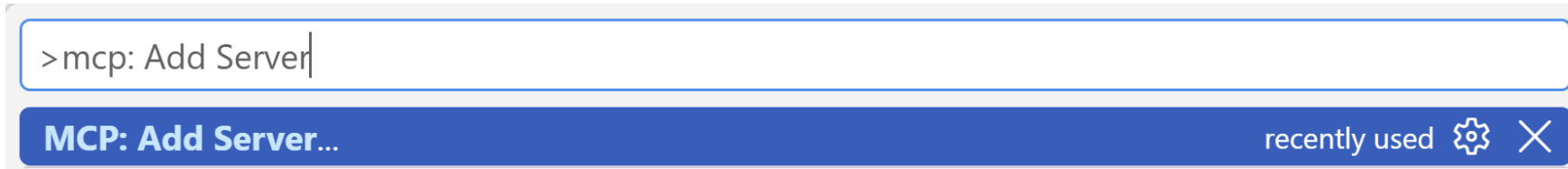
Using the MCP Server from chat view

- Prompt: *Search Microsoft Learn for documentation on Copilot Spaces and provide the source.*



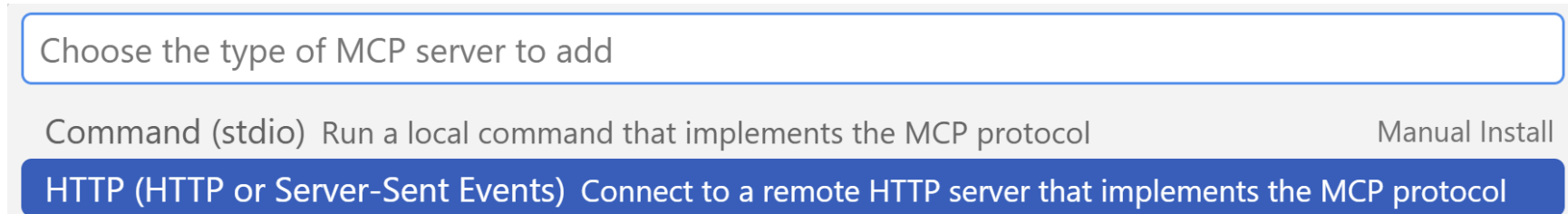
Install from command palette guided workflow

- *MCP: Add Server*



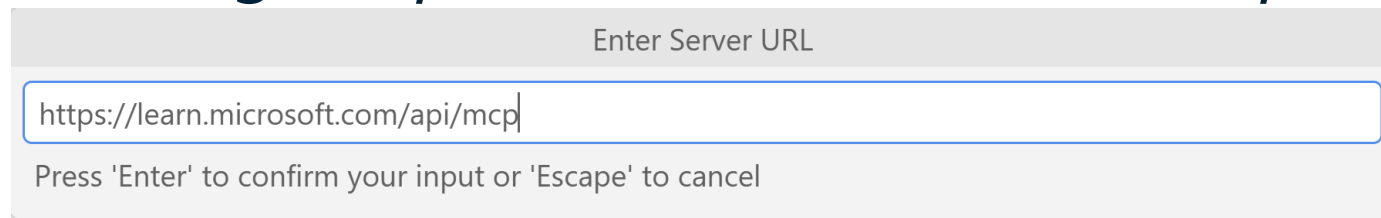
A screenshot of the Visual Studio Code command palette. The search bar contains the text ">mcp: Add Server". Below the search bar, a single command is listed: "MCP: Add Server...". To the right of this command, there are two icons: a gear icon labeled "recently used" and a close icon (an 'X' inside a circle).

- Choose *Http...*



A screenshot of a dialog box titled "Choose the type of MCP server to add". It contains two options. The first option is "Command (stdio) Run a local command that implements the MCP protocol", with the text "Manual Install" to its right. The second option, "HTTP (HTTP or Server-Sent Events) Connect to a remote HTTP server that implements the MCP protocol", is highlighted with a blue background.

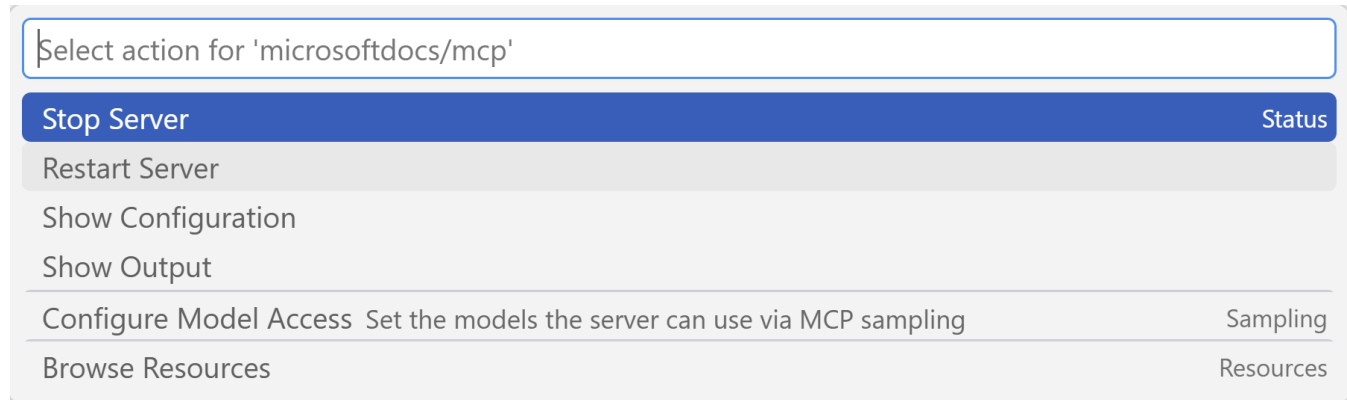
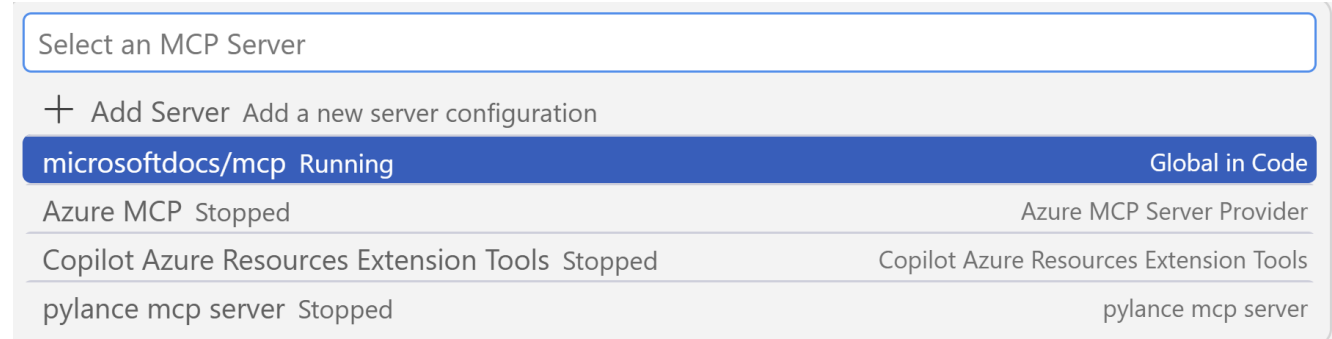
- Specify server url (e.g., *<https://learn.microsoft.com/api/mcp>*)



A screenshot of a dialog box titled "Enter Server URL". It features a text input field containing the URL "https://learn.microsoft.com/api/mcp". Below the input field, a message reads: "Press 'Enter' to confirm your input or 'Escape' to cancel".

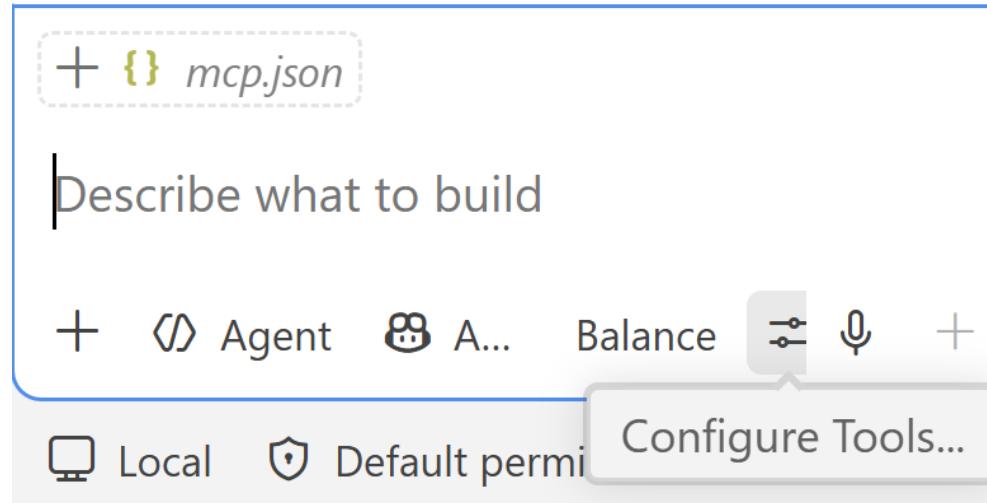
Manage MCP Servers with command palette

- *MCP: List Servers*
 - Display the MCP servers
 - Verify server is running
 - Select a server
- Right click
 - Manage the selected server
 - Can control whether server is running



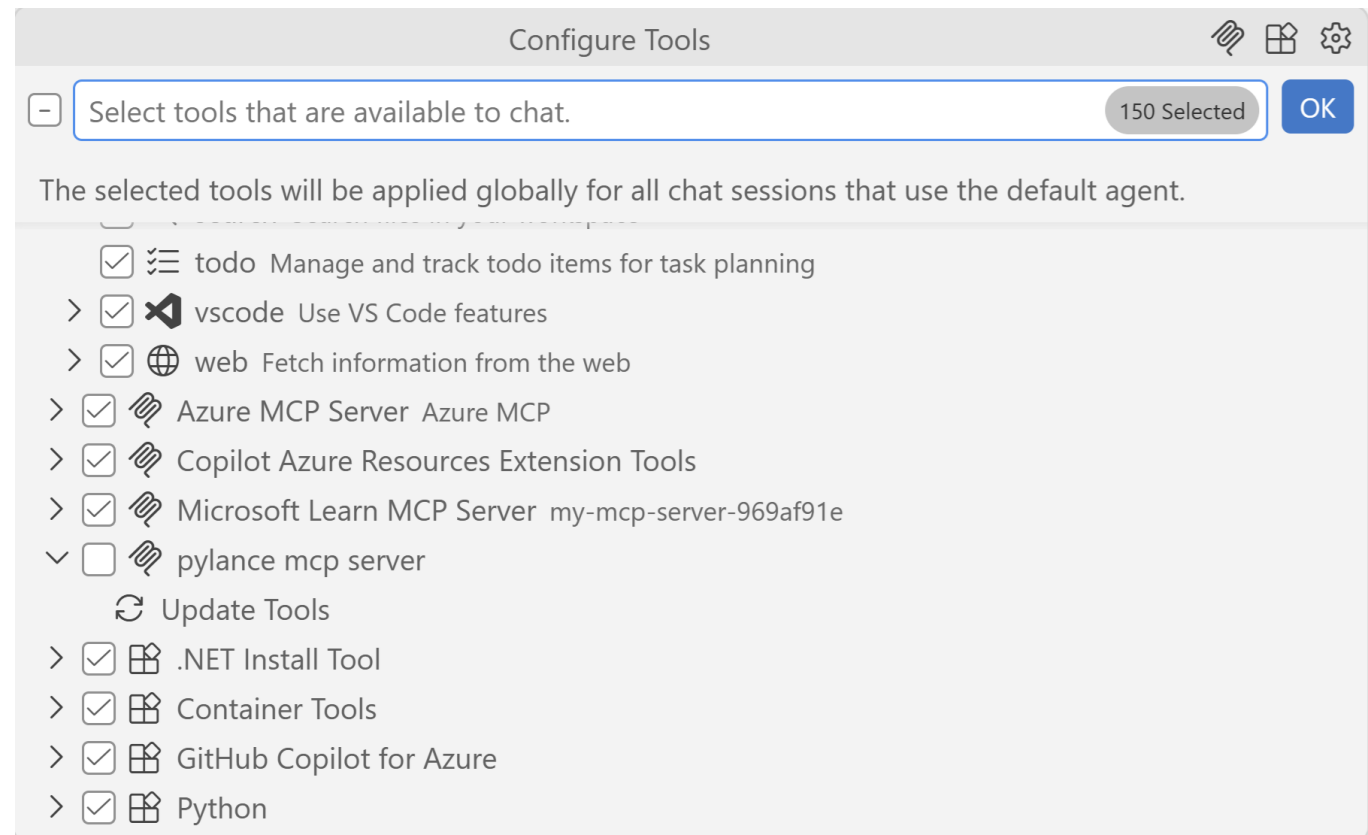
MCP Server tool availability

- Switch to agent mode in Chat View
- Click *Configure Tools...* button



Configure tools list

- Verify Microsoft Learn MCP server is available to Copilot as a tool
- Disable an MCP server
 - Turn off the checkbox
 - Tool unavailable to Copilot
 - Does not stop MCP server

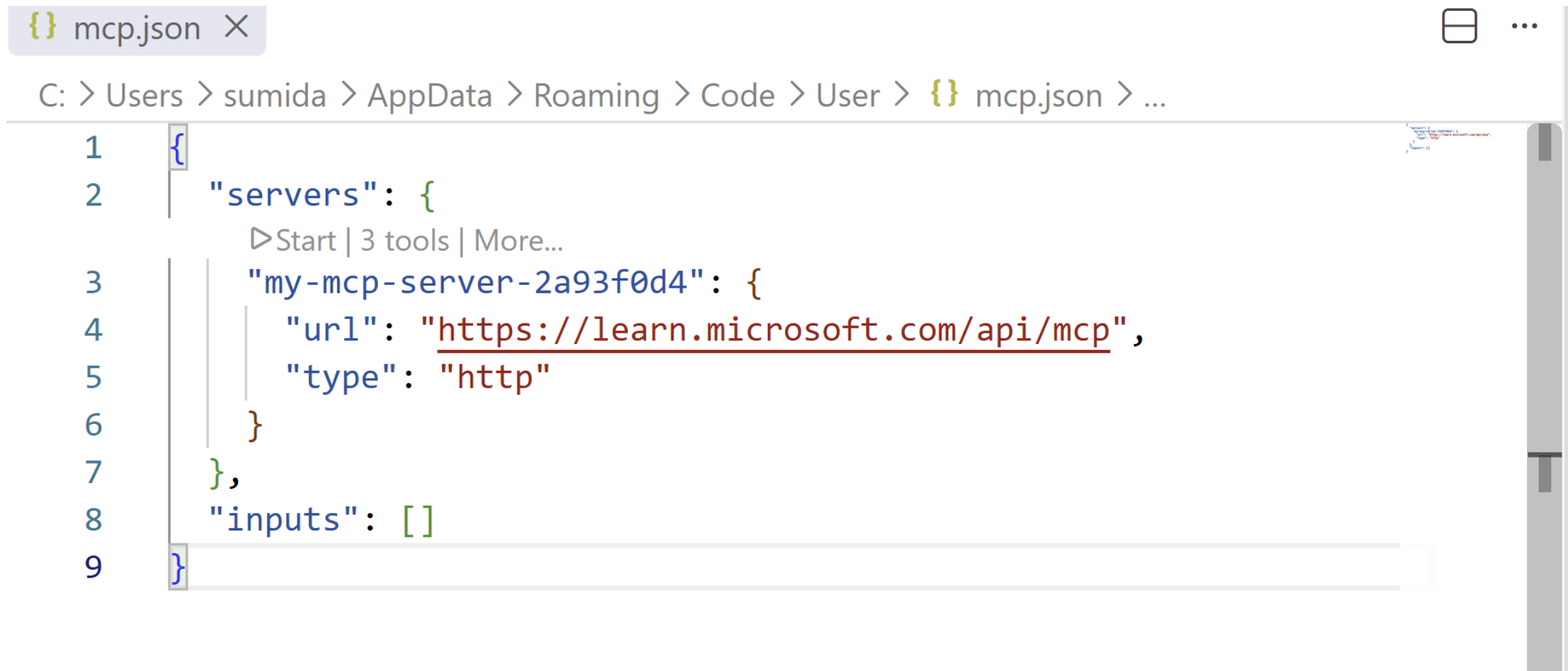


mcp.json

- Contains information about installed MCP servers
 - Manually modify to configure MCP servers (e.g., remove an MCP server)
- User profile
 - Available in all workspaces
 - `%APPDATA%\Code\User` folder
 - Command Palette -> *MCP: Open User Configuration*
 - Updated with *Install* button on MCP Server extension details page
- Workspace
 - Available only in current workspace
 - `.vscode` folder
 - Command Palette -> *MCP: Open Workspace Folder MCP Configuration*
 - Updated with *Install in Workspace* button on MCP Server extension details page

mcp.json example

- Can start the server from button in the editor



```
{} mcp.json X
```

C: > Users > sumida > AppData > Roaming > Code > User > {} mcp.json > ...

```
1 {
2   "servers": {
3     ▶ Start | 3 tools | More...
4     "my-mcp-server-2a93f0d4": {
5       "url": "https://learn.microsoft.com/api/mcp",
6       "type": "http"
7     }
8   },
9   "inputs": []
}
```

MCP server capabilities

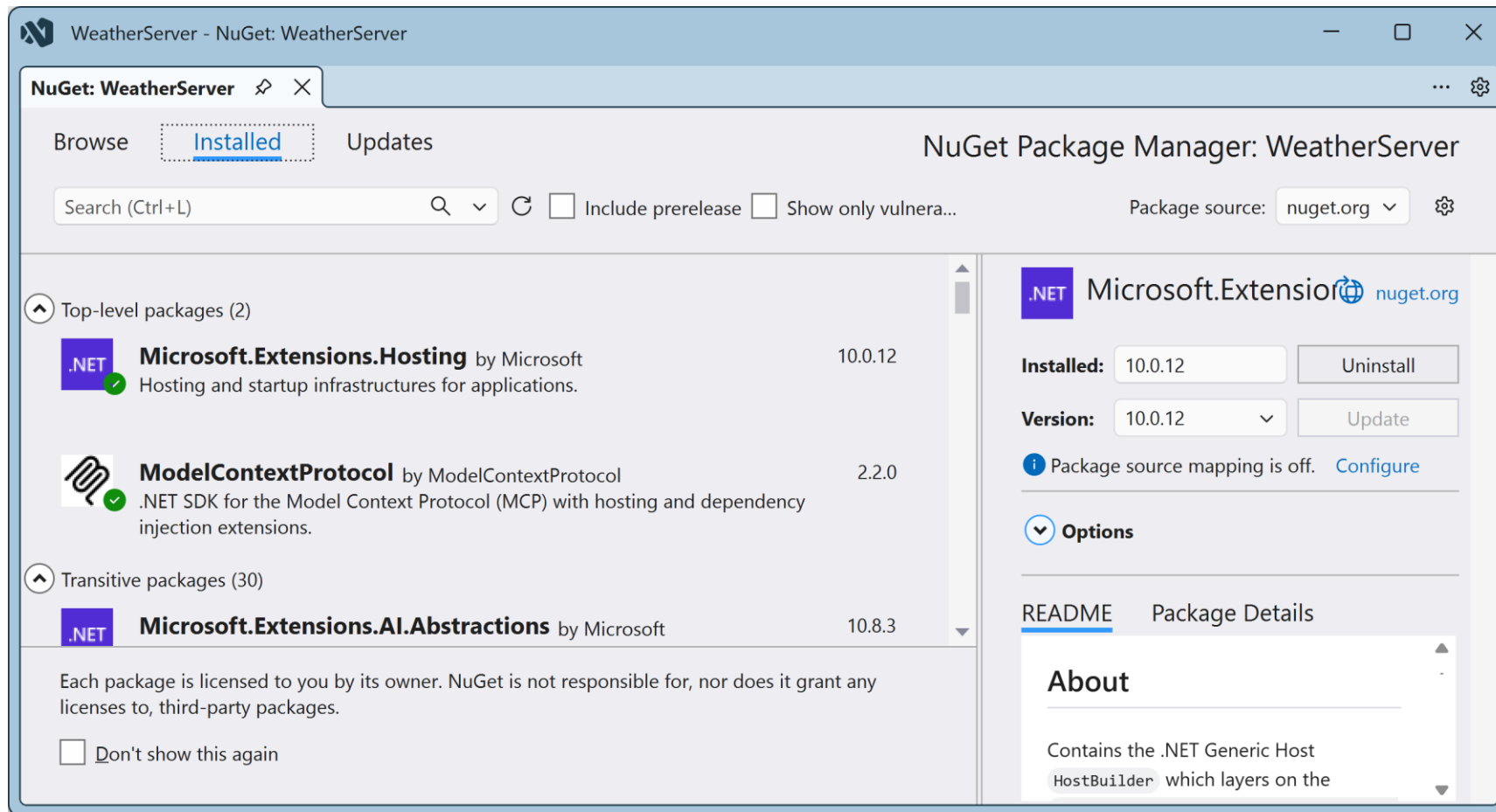
- Resources
 - Provide data consumed by clients
 - API responses
 - File contents
- Tools
 - Provide functions that can be invoked by LLM
 - E.g. Weather server
 - Requires approval before invocation
- Prompts
 - Templates for specific scenarios

Build an MCP Server

- Tool example
 - Build a weather server that retrieves data from National Weather Service
 - <https://modelcontextprotocol.io/docs/2026-07-28/develop/build-server#c%23>
 - Connect to it from VS Code
- Create MCP Server Tools
 - Retrieve data from <https://api.weather.gov>
 - *GetAlerts*
 - Retrieves active weather alerts for US state code
 - *GetForecast*
 - Retrieves the daily and extended forecast for a geographic point

Add NuGet packages

- MCP SDK and hosting



WeatherTools class

```
using ModelContextProtocol.Server;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Globalization;
using System.Text;
using System.Text.Json;
using WeatherServer;
namespace QuickstartWeatherServer.Tools;

/// <summary>
/// Provides MCP tools for retrieving weather forecasts and active alerts from the National
Weather Service.
/// </summary>
[McpServerToolType]
public static class WeatherTools
{
    // Code for GetAlerts and GetForecast
}
```

GetAlerts

```
[McpServerTool, Description("Get weather alerts for a US state code.")]
public static async Task<string> GetAlerts(
    HttpClient client,
    [Description("The US state code to get alerts for.")] string state)
{
    using var jsonDocument = await client.ReadJsonDocumentAsync($"/alerts/active/area/{state}");
    var jsonElement = jsonDocument.RootElement;
    var alerts = jsonElement.GetProperty("features").EnumerateArray();
    if (!alerts.Any()) {
        return "No active alerts for this state.";
    }
    return string.Join("\n--\n", alerts.Select(alert => {
        JsonElement properties = alert.GetProperty("properties");
        return $"""
            Event: {properties.GetProperty("event").GetString()}
            Area: {properties.GetProperty("areaDesc").GetString()}
            Severity: {properties.GetProperty("severity").GetString()}
            Description: {properties.GetProperty("description").GetString()}
            Instruction: {properties.GetProperty("instruction").GetString()}
            """;
    }));
}
```

GetForecast

```
public static async Task<string> GetForecast(
    HttpClient client,
    [Description("Latitude of the location.")] double latitude,
    [Description("Longitude of the location.")] double longitude)
{
    var pointUrl = string.Create(CultureInfo.InvariantCulture, $"/points/{latitude},{longitude}");
    using var jsonDocument = await client.ReadJsonDocumentAsync(pointUrl);
    var forecastUrl =
        jsonDocument.RootElement.GetProperty("properties").GetProperty("forecast").GetString()
        ?? throw new Exception($"No forecast URL provided by
{client.BaseAddress}points/{latitude},{longitude}");

    using var forecastDocument = await client.ReadJsonDocumentAsync(forecastUrl);
    var periods =
forecastDocument.RootElement.GetProperty("properties").GetProperty("periods").EnumerateArray();
    return string.Join("\n---\n", periods.Select(period => $""
        {period.GetProperty("name").GetString()}
        Temperature: {period.GetProperty("temperature").GetInt32()}°F
        Wind: {period.GetProperty("windSpeed").GetString()}
{period.GetProperty("windDirection").GetString()}
        Forecast: {period.GetProperty("detailedForecast").GetString()}
        ""));
}
```

HttpClient extension method

```
using System.Text.Json;
namespace WeatherServer
{
    /// <summary>
    /// Sends an HTTP GET request and parses the JSON response into a JsonDocument />.
    /// </summary>
    internal static class HttpClientExt
    {
        public static async Task<JsonDocument> ReadJsonDocumentAsync(
            this HttpClient client, string requestUri)
        {
            using var response = await client.GetAsync(requestUri);
            response.EnsureSuccessStatusCode();
            return await JsonDocument.ParseAsync(
                await response.Content.ReadAsStreamAsync());
        }
    }
}
```

Program.cs

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using ModelContextProtocol;
using System.Net.Http.Headers;

// Create an empty host builder for the MCP server application.
var builder = Host.CreateEmptyApplicationBuilder(settings: null);

// Configure the MCP server to listen over standard input/output and register
// all tool implementations discovered in the current assembly.
builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

// Continued on next slide
```

Program.cs continued

```
// Register a shared HttpClient configured for the National Weather Service API.  
// The user-agent header identifies this server when making outbound requests.  
builder.Services.AddSingleton(_ =>  
{  
    var client = new HttpClient() { BaseAddress = new Uri("https://api.weather.gov") };  
    client.DefaultRequestHeaders.UserAgent.Add(new ProductInfoHeaderValue("weather-tool",  
"1.0"));  
    return client;  
});  
  
var app = builder.Build();  
  
// Run the MCP server application.  
await app.RunAsync();
```

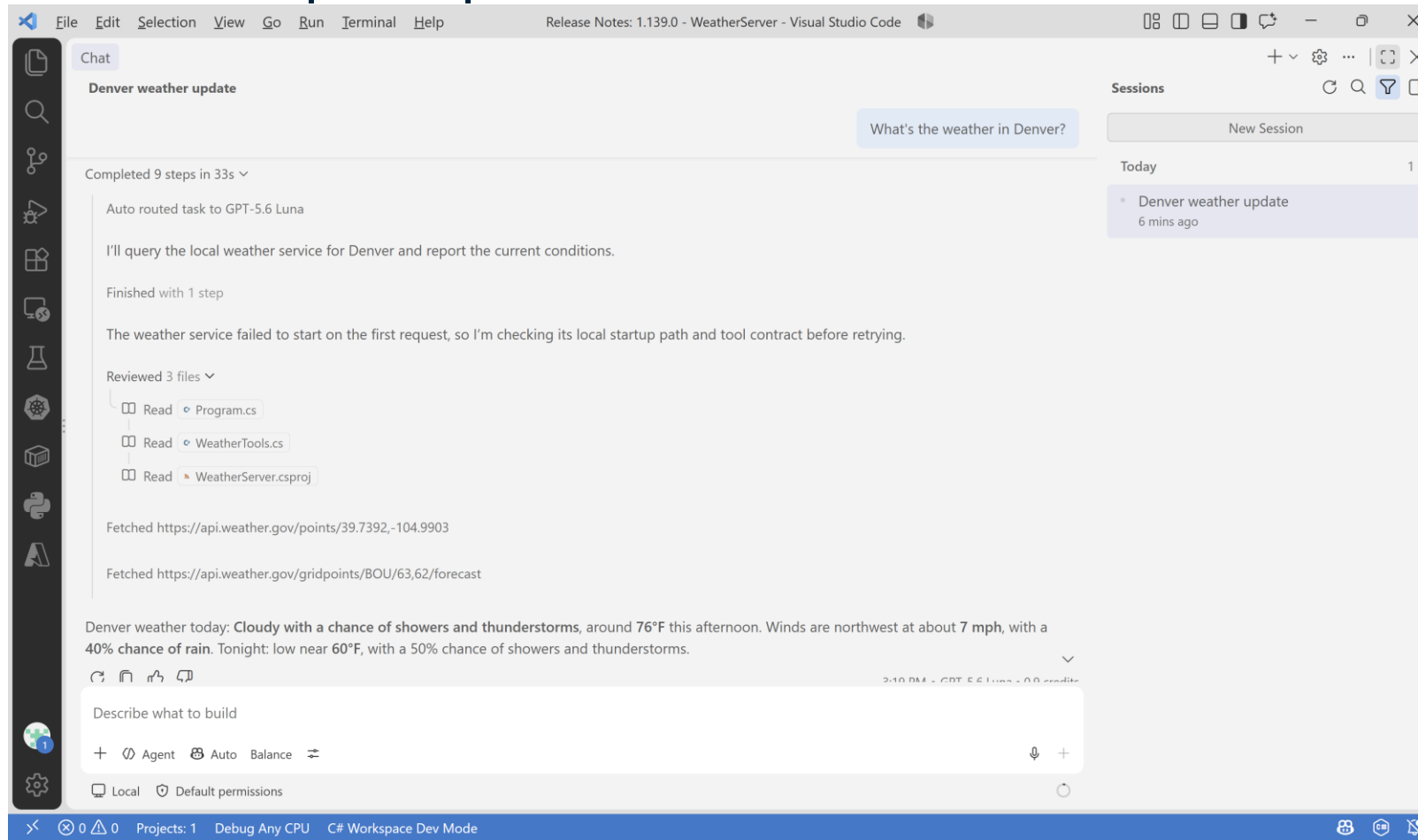
Connecting the WeatherServer to VS Code

- Command palette – *MCP: Open User Configuration*
 - Add MCP Server to the *mcp.json* file

```
{  
  "servers": {  
    "weather": {  
      "type": "stdio",  
      "command": "dotnet",  
      "args": [  
        "run",  
        "--project",  
        "C:\\\\demos\\\\WeatherServer\\\\WeatherServer\\\\WeatherServer.csproj"  
      ]  
    }  
  }  
}
```

Using the Weather MCP Server

- Start the server and prompt: *What's the weather in Denver?*



Debug an MCP Server

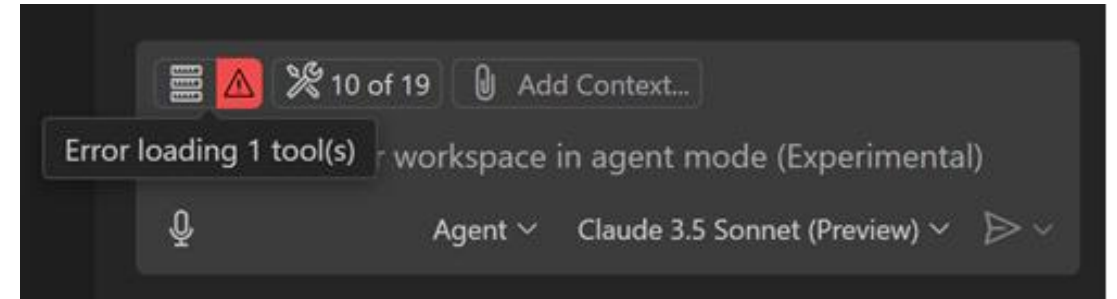
- Tools
 - MCP Inspector
 - <https://github.com/modelcontextprotocol/inspector>
 - Interactive developer tool
 - Invoke tools, prompts, and resources
 - Can observe notification stream
 - Server Logging
 - Stderr
 - OpenTelemetry
 - <https://opentelemetry.io/>

Debug an MCP Server in VS Code

- MCP development mode
 - *dev* key
 - Add to *mcp.json* file to enable
 - *watch* property
 - Glob pattern that watches for changes that restart MCP server
 - *debug* property
 - Configures debugger for MCP server
 - Supported servers
 - Node.js
 - Python
- <https://code.visualstudio.com/api/extension-guides/ai/mcp#troubleshoot-and-debug-mcp-servers>

VS Code MCP Output log

- Error indicator in the Chat view
 - Click on error notification
 - Click on *Show Output*
- Command palette – MCP: List Servers
 - Select server
 - Click on *Show Output*



```
... Filter MCP: my-remote-serve ...  
2025-03-24 10:47:52.181 [info] Starting server my-remote-server  
2025-03-24 10:47:52.182 [info] Connection state: Starting  
2025-03-24 10:47:52.194 [info] Starting server from LocalProcess extension host  
2025-03-24 10:47:52.218 [info] Connection state: Error Error connecting to http://localhost:3000/  
: TypeError: fetch failed
```

Lab 3 – MCP

- Integrate MCP with Copilot
 - <https://github.com/skills/integrate-mcp-with-copilot/tree/main>

Summary and conclusions

- GitHub Copilot supports connecting to custom knowledge/tools
 - Spaces
 - Provides control over context used by GitHub Copilot
 - MCP
 - Open standard for connecting AI systems to tools and data
 - Use cases
 - Integrate with tools
 - Retrieve additional context
- Tool-enabled AI represents the next generation beyond LLMs